

Plan de implementación — Aplicación PWA de hábitos

1. Objetivo

Construir una aplicación web/PWA de hábitos, mobile-first, simple y rápida. La aplicación permitirá:

- Crear, editar, activar, pausar y eliminar hábitos.
- Definir la frecuencia de cada hábito.
- Registrar el cumplimiento de cada ocurrencia.
- Consultar progreso diario.
- Consultar estadísticas e informes semanales y mensuales.
- Gestionar cuentas de usuario mediante autenticación.
- Mantener los datos aislados por usuario.
- Ser instalable como PWA en dispositivos móviles y escritorio.
- Preparar una arquitectura que pueda evolucionar sin introducir complejidad innecesaria.

El agente de implementación es **Lia (Hermes)**.

2. Stack tecnológico

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- shadcn/ui

PWA

- vite-plugin-pwa
- Web App Manifest
- Service Worker
- Diseño responsive y mobile-first

Backend / datos

- Supabase
- PostgreSQL
- Supabase Auth
- Row Level Security (RLS)
- `@supabase/supabase-js`

Deployment

- GitHub
- Vercel

Agente

- Lia / Hermes

Integración del agente con Supabase

- Supabase MCP
- Utilizar inicialmente el proyecto de desarrollo.
- No conectar el agente con producción con permisos de escritura durante la fase inicial.

3. Principios de arquitectura

1. Mantener la aplicación simple.
2. No crear un backend Node/Express propio salvo que exista una necesidad técnica real.
3. Supabase será la fuente de verdad de los datos.
4. Toda información perteneciente a un usuario debe estar protegida mediante RLS.
5. Nunca exponer claves secretas o `service\_role` en el frontend.
6. Los cambios de base de datos deben realizarse mediante migraciones versionadas.
7. Separar desarrollo y producción.
8. No añadir dependencias innecesarias.
9. Priorizar rendimiento y UX móvil.
10. La interfaz debe permitir registrar un hábito con la menor cantidad posible de pasos.
11. La lógica de frecuencia debe estar separada de la presentación.
12. Las estadísticas deben derivarse de los registros, evitando duplicar datos innecesariamente.

4. Modelo funcional

4.1 Usuario

Cada usuario autenticado tendrá:

- Identidad gestionada por Supabase Auth.
- Perfil opcional asociado a `auth.users`.
- Sus propios hábitos.
- Sus propios registros.
- Sus propias estadísticas.

Un usuario nunca debe poder leer o modificar datos pertenecientes a otro usuario.

5. Modelo de hábitos

Un hábito debe soportar diferentes tipos de frecuencia.

Ejemplos:

Una vez al día

- Ejercicio
- Leer
- Meditar

Varias veces al día

- Beber agua 6 veces al día.

Intervalo de horas

- Cada 6 horas.
- Cada 8 horas.
- Cada 12 horas.

Días específicos

- Gimnasio lunes, miércoles y viernes.

Cantidad objetivo

- Beber 2 litros.
- Leer 30 minutos.
- Hacer 50 repeticiones.

La arquitectura debe permitir ampliar posteriormente otros tipos de hábitos sin romper los existentes.

6. Esquema inicial de base de datos

6.1 `profiles`

Perfil asociado al usuario de Supabase Auth.

Campos sugeridos:

- `id` UUID — PK y referencia a `auth.users.id`
- `display\_name` TEXT

- `avatar\_url` TEXT nullable
- `timezone` TEXT
- `created\_at` TIMESTAMPTZ
- `updated\_at` TIMESTAMPTZ

La zona horaria del usuario es importante para determinar correctamente cuándo comienza y termina su día.

6.2 `habits`

Representa la definición de un hábito.

Campos sugeridos:

- `id` UUID — PK
- `user\_id` UUID — FK a `profiles.id`
- `name` TEXT
- `description` TEXT nullable
- `category` TEXT nullable
- `icon` TEXT nullable
- `color` TEXT nullable
- `frequency\_type` TEXT
- `frequency\_config` JSONB
- `target\_type` TEXT nullable
- `target\_value` NUMERIC nullable
- `unit` TEXT nullable
- `start\_date` DATE
- `end\_date` DATE nullable
- `is\_active` BOOLEAN
- `created\_at` TIMESTAMPTZ
- `updated\_at` TIMESTAMPTZ

`frequency\_type`

Inicialmente considerar:

- `daily`
- `weekly`
- `interval`
- `multiple\_daily`

La implementación debe evitar hardcodear la lógica directamente en múltiples componentes.

`frequency\_config`

Usar JSONB para almacenar la configuración específica de cada frecuencia.

Ejemplos conceptuales:

```
{
  "times_per_day": 1
}

{
  "interval_hours": 6
}

{
  "days_of_week": [1, 3, 5],
  "times_per_day": 1
}
```

No implementar formatos adicionales hasta que exista una necesidad real.

7. Registros de cumplimiento

7.1 `habit\_occurrences`

Representa cada ocurrencia programada de un hábito.

Campos sugeridos:

- `id` UUID — PK
- `habit\_id` UUID — FK
- `user\_id` UUID — FK
- `scheduled\_at` TIMESTAMPTZ
- `completed\_at` TIMESTAMPTZ nullable
- `status` TEXT
- `value` NUMERIC nullable
- `note` TEXT nullable
- `created\_at` TIMESTAMPTZ
- `updated\_at` TIMESTAMPTZ

Estados iniciales

- `pending`
- `completed`
- `skipped`
- `missed`

La arquitectura debe permitir calcular:

- cumplimiento diario
- cumplimiento semanal
- cumplimiento mensual

- rachas
- hábitos más consistentes
- hábitos con menor cumplimiento

8. Generación de ocurrencias

No generar millones de registros futuros.

La aplicación debe generar o resolver las ocurrencias necesarias para el período que se está consultando.

Por ejemplo:

- Hoy.
- Próximos días.
- Período solicitado por estadísticas.

Si se determina que la generación persistente de ocurrencias es más conveniente, justificarlo antes de implementarlo.

La lógica debe respetar la zona horaria del usuario.

9. Estadísticas

No crear inicialmente una tabla de estadísticas duplicada.

Calcular las métricas a partir de `habit\_occurrences` y `habits`.

Métricas iniciales:

- Hábitos completados hoy.
- Porcentaje de cumplimiento diario.
- Porcentaje de cumplimiento semanal.
- Porcentaje de cumplimiento mensual.
- Racha actual.
- Mejor racha.
- Total de hábitos completados.
- Cumplimiento por hábito.
- Cumplimiento por día.
- Tendencia semanal.
- Tendencia mensual.

Si posteriormente el volumen de datos justifica materializar estadísticas, evaluarlo mediante consultas optimizadas, vistas o funciones PostgreSQL.

10. Autenticación

Usar exclusivamente Supabase Auth para la primera versión.

Primera versión:

- Registro con email y contraseña.
- Login.
- Logout.
- Persistencia de sesión.
- Recuperación de contraseña.
- Protección de rutas privadas.

Preparar la arquitectura para añadir posteriormente OAuth/Google si se desea.

11. Seguridad

RLS

Activar RLS en todas las tablas que contengan datos de usuarios.

Regla fundamental:

Un usuario solo puede `SELECT`, `INSERT`, `UPDATE` y `DELETE` sobre sus propios datos.

Implementar políticas RLS basadas en el usuario autenticado.

No confiar únicamente en filtros realizados en React.

La seguridad debe existir en PostgreSQL.

12. Variables de entorno

Usar variables de entorno para Supabase.

Frontend:

```
VITE_SUPABASE_URL=  
VITE_SUPABASE_PUBLISHABLE_KEY=
```

No colocar nunca:

```
SUPABASE_SERVICE_ROLE_KEY=
```

en variables `VITE\_\*` ni exponer claves secretas al navegador.

Las variables de producción deberán configurarse en Vercel.

13. PWA

Implementar:

- Manifest.
- Nombre y nombre corto.
- Iconos.
- `theme\_color`.
- `background\_color`.
- `display: standalone`.
- Service Worker.
- Instalabilidad.
- Página/fallback apropiado para errores de red.

Prioridad:

1. Experiencia móvil.
2. Carga rápida.
3. Instalación sencilla.
4. Interfaz usable con una mano.

No implementar offline completo en la primera versión salvo que exista una necesidad explícita.

14. Offline

Primera versión:

- La aplicación puede depender de conexión para sincronizar con Supabase.
- El Service Worker puede cachear recursos estáticos.

No introducir Dexie/IndexedDB ni sincronización bidireccional todavía.

Futura mejora:

```
Usuario registra hábito
  ↓
IndexedDB local
  ↓
Cola de sincronización
  ↓
Supabase
```

Solo implementar esto después de validar que realmente se necesita.

15. Estructura de interfaz

Navegación principal

Mobile:

- Hoy
- Hábitos
- Estadísticas
- Ajustes

Desktop puede utilizar sidebar.

Pantalla Hoy

Debe ser la pantalla principal.

Mostrar:

- Fecha actual.
- Progreso del día.
- Hábitos pendientes.
- Hábitos completados.
- Próxima ocurrencia.
- Acción rápida para marcar como completado.

Objetivo:

Registrar un hábito en uno o dos toques.

Pantalla Hábitos

Permitir:

- Crear.
- Editar.
- Pausar.
- Reactivar.
- Eliminar.
- Ver configuración.
- Ver historial individual.

Crear hábito

Flujo simple:

1. Nombre.
2. Frecuencia.

3. Horario/configuración.
4. Objetivo opcional.
5. Fecha de inicio.
6. Guardar.

No mostrar configuraciones avanzadas innecesarias por defecto.

Estadísticas

Mostrar:

Semana

- Cumplimiento.
- Racha.
- Hábitos completados.
- Evolución diaria.

Mes

- Cumplimiento.
- Comparación por semanas.
- Hábitos más consistentes.
- Hábitos con menor cumplimiento.

Usar gráficos simples y legibles.

16. Diseño visual

La interfaz debe ser:

- Minimalista.
- Clara.
- Moderna.
- Mobile-first.
- Accesible.
- Con suficiente contraste.
- Con estados visuales claros.
- Sin exceso de animaciones.

Evitar convertir la aplicación en un dashboard saturado.

La información más importante debe ser visible inmediatamente.

17. Rendimiento

Objetivos:

- Bundle razonablemente pequeño.
- Lazy loading de páginas secundarias si aporta beneficio.
- Evitar dependencias grandes innecesarias.
- Consultas Supabase eficientes.
- Índices PostgreSQL para campos usados frecuentemente.
- No descargar todo el historial cuando solo se necesita el día actual.
- Utilizar paginación o rangos temporales para historiales grandes.

18. Índices iniciales

Evaluar índices para:

- ``habits.user_id``
- ``habit_occurrences.user_id``
- ``habit_occurrences.habit_id``
- ``habit_occurrences.scheduled_at``
- combinaciones frecuentes como ``(user_id, scheduled_at)``

Crear únicamente índices justificados por las consultas reales.

19. Git y estructura del proyecto

Mantener el código organizado.

Estructura orientativa:

```
src/  
  components/  
  features/  
    auth/  
    habits/  
    occurrences/  
    dashboard/  
    statistics/  
  layouts/  
  lib/  
    supabase/  
    utils/  
  pages/  
  hooks/  
  types/  
  main.tsx
```

La estructura puede modificarse si existe una razón técnica.

Evitar sobrearquitectura.

20. TypeScript

Usar TypeScript estrictamente.

Generar tipos de Supabase a partir del esquema de PostgreSQL.

No duplicar manualmente los tipos de las tablas si pueden generarse automáticamente.

21. Migraciones

Toda modificación estructural de PostgreSQL debe quedar versionada.

No depender de cambios manuales no registrados en el dashboard de Supabase.

Flujo:

```
graph TD;
  Cambio --> Migration;
  Migration --> Git;
  Git --> Review;
  Review --> Supabase;
```

Lia debe evitar modificaciones destructivas en producción sin confirmación explícita.

22. Entornos

Mantener como mínimo:

```
Supabase DEV
Supabase PROD
```

Lia trabaja inicialmente con DEV.

Vercel deberá utilizar las credenciales correspondientes al entorno configurado.

No mezclar datos de desarrollo y producción.

23. Uso de Supabase MCP

Configurar MCP inicialmente limitado al proyecto de desarrollo.

Prioridades:

1. Consultar esquema.
2. Crear migraciones.
3. Crear tablas.
4. Crear índices.
5. Configurar RLS.
6. Generar tipos.
7. Probar consultas.
8. Revisar errores.

No permitir que Lia modifique producción automáticamente.

Antes de ejecutar cambios destructivos:

- explicar qué se va a cambiar;
- identificar los datos afectados;
- solicitar confirmación humana si existe riesgo de pérdida.

24. Vercel

Vercel será únicamente el hosting/deployment del frontend inicialmente.

Configurar:

- Proyecto conectado a GitHub.
- Deploy automático.
- Variables de entorno.
- Preview deployments.
- Production deployment.

Cada Pull Request debería poder generar un preview.

25. Git workflow

Recomendación:

```
main
↓
production

feature/*
↓
Pull Request
↓
Preview Vercel
↓
Review
↓
merge
↓
production
```

Lia no debe hacer cambios masivos directamente en `main` sin revisar el resultado.

26. Fases de implementación

Fase 1 — Inspección

Lia debe:

- Analizar el repositorio existente.
- Identificar qué está instalado.
- Identificar el estado actual de Vite/React.
- Revisar configuración de Vercel.
- Revisar configuración existente de PWA.
- No modificar código todavía.

Entregar primero un diagnóstico.

Fase 2 — Fundación

Implementar:

- TypeScript.
- Tailwind.
- shadcn/ui.
- Estructura base.
- Supabase client.
- Variables de entorno.
- Routing.
- Layout.
- PWA.

Fase 3 — Supabase

Crear mediante migraciones:

- `profiles`
- `habits`
- `habit\_occurrences`

Configurar:

- Relaciones.
- Constraints.
- Índices.
- RLS.
- Políticas.

Generar tipos TypeScript.

Fase 4 — Auth

Implementar:

- Registro.
- Login.
- Logout.
- Recuperación de contraseña.
- Protección de rutas.
- Persistencia de sesión.

Fase 5 — Hábitos

Implementar:

- Crear hábito.
- Editar.
- Pausar/reactivar.
- Eliminar.
- Configurar frecuencia.
- Configurar objetivo.
- Validaciones.

Fase 6 — Registro diario

Implementar:

- Vista Hoy.
- Ocurrencias.
- Completar.
- Deshacer.
- Omitir.

- Estado pendiente.
- Estado completado.
- Manejo de horarios.

Fase 7 — Estadísticas

Implementar:

- Resumen diario.
- Informe semanal.
- Informe mensual.
- Rachas.
- Porcentajes.
- Gráficos.

Fase 8 — PWA y UX

Verificar:

- Instalación.
- Iconos.
- Manifest.
- Service Worker.
- Responsive.
- Touch targets.
- Accesibilidad.
- Loading states.
- Empty states.
- Error states.

Fase 9 — Testing

Como mínimo:

Unit tests

- Cálculo de frecuencias.
- Generación de ocurrencias.
- Cálculo de porcentajes.
- Cálculo de rachas.

Integration tests

- Auth.
- CRUD de hábitos.
- Registro de ocurrencias.
- RLS.

E2E

Flujo principal:

```
Registro
↓
Login
↓
Crear hábito
↓
Ver hábito en Hoy
↓
Completar
↓
Consultar estadísticas
```

Fase 10 — Deployment

Antes de producción:

- Build limpio.
- Variables correctas.
- Supabase PROD configurado.
- RLS verificado.
- No existen claves secretas en frontend.
- PWA instalable.
- Vercel production deployment.
- Smoke test completo.

27. Criterios de aceptación

La primera versión se considera funcional cuando un usuario puede:

1. Registrarse.
2. Iniciar sesión.
3. Crear un hábito diario.
4. Crear un hábito con múltiples ocurrencias.
5. Crear un hábito con intervalo horario.
6. Configurar un hábito para determinados días.
7. Ver las ocurrencias correspondientes al día actual.

8. Marcar una ocurrencia como completada.
9. Deshacer el registro.
10. Consultar su progreso.
11. Consultar estadísticas semanales.
12. Consultar estadísticas mensuales.
13. Ver su racha.
14. Editar un hábito.
15. Pausar un hábito.
16. Reactivar un hábito.
17. Eliminar un hábito.
18. Instalar la aplicación como PWA.
19. Acceder desde móvil y escritorio.
20. No poder acceder a datos de otro usuario.

28. Reglas para Lia

Lia debe seguir estas reglas durante el desarrollo:

Antes de modificar

- Inspeccionar primero.
- Entender la arquitectura existente.
- No asumir que una dependencia es necesaria.
- No reemplazar tecnologías sin justificación.

Durante el desarrollo

- Cambios pequeños y verificables.
- Mantener TypeScript estricto.
- Mantener código modular.
- Evitar duplicación.
- Validar consultas.
- Mantener RLS activo.

Antes de instalar una dependencia

Preguntar:

¿Realmente necesitamos esta dependencia o podemos resolverlo con las herramientas existentes?

Antes de cambiar la base de datos

- Crear migration.
- Revisar impacto.

- Evitar operaciones destructivas.
- Probar en DEV.

Antes de producción

- Ejecutar tests.
- Ejecutar build.
- Revisar variables de entorno.
- Revisar RLS.
- Revisar seguridad.
- Revisar PWA.

29. Lo que NO debe hacer Lia

No debe:

- Crear un backend innecesario.
- Crear un servidor PostgreSQL local como sustituto de Supabase.
- Introducir Firebase.
- Introducir MongoDB.
- Introducir Prisma salvo necesidad técnica demostrada.
- Exponer `service\_role`.
- Desactivar RLS para solucionar errores.
- Crear datos duplicados innecesariamente.
- Generar ocurrencias infinitas hacia el futuro.
- Añadir offline complejo en la primera versión.
- Modificar producción de forma destructiva sin confirmación.
- Añadir librerías grandes sin justificación.
- Priorizar la estética sobre la usabilidad.
- Construir un dashboard excesivamente complejo.

30. Primera instrucción para Lia

Antes de escribir o modificar código, realizar exclusivamente una fase de análisis.

Entregar:

1. Estado actual del repositorio.
2. Stack existente.
3. Dependencias existentes.
4. Qué debe conservarse.

5. Qué debe instalarse.
6. Arquitectura propuesta.
7. Esquema de base de datos propuesto.
8. Políticas RLS propuestas.
9. Plan de implementación por fases.
10. Riesgos técnicos detectados.

No ejecutar cambios destructivos ni crear producción todavía.

Después del análisis, esperar confirmación humana para comenzar la implementación.

31. Resultado esperado

El resultado final debe ser una PWA de hábitos:

- rápida;
- simple;
- instalable;
- responsive;
- segura;
- mantenible;
- preparada para múltiples usuarios;
- con autenticación;
- con PostgreSQL;
- con estadísticas;
- con soporte para diferentes frecuencias;
- desplegada mediante Vercel;
- versionada mediante GitHub;
- administrable durante el desarrollo mediante Lia + Supabase MCP.

La prioridad es **simplicidad + seguridad + buena experiencia móvil**, no añadir funcionalidades innecesarias.